Building a Simple ETL Pipeline

Components of Data Pipelines

MATH 153 | Data Wrangling / Data Engineering | Howard University, Department of Mathematics

Moussa Doumbia, Ph.D.

Files for this lesson

`ETL_Pipeline_Lecture.ipynb`	the worked example, ready to run
`ETL_Activity.ipynb`	the in-class activity (with blanks to fill in)
`ETL_Activity_SOLUTIONS.ipynb`	the answer key
`data/`	café sales: 3 daily files + `menu.csv`
`activity_data/`	library checkouts: 3 daily files + `books.csv`

1. What is a data pipeline?

> A **data pipeline** is a series of steps that moves data from where it *is* to where it is *useful*, doing the same thing every time it runs.

The important words are **every time it runs**. A notebook you run once by hand is an analysis. A pipeline runs tomorrow, and next Tuesday, without you.

The five components

#	Component	What it means	In our example
1	**Source**	where the data comes from	3 daily CSV files from the café till
2	**Extract**	read it in	`pd.read_csv`
3	**Transform**	make it usable	fix prices, fix names, join the menu
4	**Load**	put it where people will use it	a SQLite table + a summary CSV
5	**Schedule**	decide when it runs	every morning at 6 a.m.

Steps 2, 3 and 4 are so common that they have a name of their own: **ETL**.

```
SOURCE       EXTRACT      TRANSFORM       LOAD        SCHEDULE
3 CSVs  --> read     --> clean+join --> cafe.db <-- daily
+ menu                                  summary      6 a.m.
```

ETL in one sentence each

- **Extract** — get the raw data, *without changing it*.
- **Transform** — clean it, reshape it, combine it, calculate from it.
- **Load** — write the result somewhere other people can read.

> You will also hear **ELT**: load the raw data into the database first, and transform it later *inside* the database. Same three letters, different order. ELT suits very large data and powerful warehouses; ETL suits what we are doing today.

2. The worked example: Bison Café sales

The café sends one sales file per day. We have three days. We want a clean table and a daily revenue report.

Look at the data first

Never write a pipeline for data you have not looked at.

```
sale_id,sale_date,sale_time,item,qty,unit_price
S-5001,2027-03-02,14:32,FRUIT CUP,2,$4.00
S-5002,2027-03-02,15:22,Turkey Wrap,2,$7.50
S-5004,2027-03-02,11:58,latte,3,$4.25
```

Two things are already wrong, and both are normal:

1. `unit_price` **is text, not a number**. It says `$4.25`. You cannot multiply by `$4.25`.
2. `item` **is spelled inconsistently** — `FRUIT CUP`, `latte`, `" Bagel "` — because the till lets staff type it by hand.

Fixing those two things is what "Transform" will mean today. There is also a reference table, `menu.csv`, giving each item's category.

E — Extract

Read the raw files. Change nothing.

```python
def extract(data_dir):
    """Read every daily sales file and stack them into one DataFrame."""
    files = sorted(data_dir.glob("sales_2027-*.csv"))
    frames = [pd.read_csv(f) for f in files]
    return pd.concat(frames, ignore_index=True)

raw = extract(DATA)        # 107 rows from 3 files
```

Remember

Extract is deliberately boring. Keeping the raw data untouched means that when something goes wrong later, you know where to start again. `ignore_index=True` renumbers the stacked rows 0, 1, 2, ... so you do not end up with three rows all numbered 0.

T — Transform

Four small steps.

Step 1 — make the price a number.

```python
df["unit_price"] = df["unit_price"].str.replace("$", "", regex=False).
    astype(float)
```

Step 2 — make the date a real date.

```python
df["sale_date"] = pd.to_datetime(df["sale_date"], format="%Y-%m-%d")
```

A date stored as text is just a string — you cannot sort it reliably, subtract it, or ask what day of the week it was.

Step 3 — clean the item names.

```
df["item"] = df["item"].str.strip().str.title()
```

```
distinct spellings before: 29
distinct item names after:  8
```

`.str.strip()` removes spaces at the ends; `.str.title()` capitalises each word, so `"LATTE"` and `"latte"` both become `"Latte"`.

Step 4 — join the menu, then calculate.

```
df = df.merge(menu, on="item", how="left")
df["revenue"] = (df["qty"] * df["unit_price"]).round(2)
```

Watch out

Step 3 had to come before step 4. `"latte"` would not have matched `"Latte"` in the menu, that sale would have had no category, and its revenue would have quietly vanished from the report.

Two checks to make after every join

Did the row count stay the same? 107 in, 107 out. If it grew, the reference table has duplicate rows and every sale got counted twice.

Did anything fail to match? Zero unmatched here. A blank category means a spelling your cleaning step missed.

L — Load

Two outputs, for two audiences: a **database table** for anyone who wants to query it, and a **summary CSV** for the manager who just wants the daily numbers.

```
con = sqlite3.connect("cafe.db")
out.to_sql("sales", con, if_exists="replace", index=False)
con.close()
summary.to_csv("daily_summary.csv", index=False)
```

The result, from 107 sales over three days:

sale_date	category	items_sold	revenue
2027-03-02	Drink	35	$119.75
2027-03-02	Food	40	$222.85
2027-03-03	Drink	48	$162.80
2027-03-03	Food	29	$112.40
2027-03-04	Drink	39	$124.30
2027-03-04	Food	29	$126.25
Total			**$868.35**

3. The whole pipeline, in one function

Everything above, in twelve lines. **This is the deliverable** — not the notebook, the function.

```
def run_pipeline(data_dir=DATA, db_path="cafe.db", csv_path="daily_summary
    .csv"):
    """Extract -> Transform -> Load. Run this every morning."""
    menu  = pd.read_csv(data_dir / "menu.csv")

    raw   = extract(data_dir)                          # E
    clean = transform(raw, menu)                       # T

    assert clean["category"].notna().all(), "some items did not match the
       menu"
    assert (clean["revenue"] > 0).all(),    "a sale has zero or negative
       revenue"

    summary = (clean
               .assign(day=clean["sale_date"].dt.date)
               .groupby(["day", "category"], as_index=False)
               .agg(items_sold=("qty", "sum"), revenue=("revenue", "sum"))
               .rename(columns={"day": "sale_date"}))

    load(clean, summary, db_path, csv_path)            # L
    return clean, summary
```

Why a function, and not just cells?

It has a name. You can tell someone "run `run_pipeline()`" instead of "open the notebook and run cells 4 through 19 but not cell 12".

It takes arguments. Point it at a different folder and it works on different data.

It can be scheduled. A computer can call a function at 6 a.m. It cannot click through a notebook.

4. Two things every pipeline needs

Run it twice

```
run_pipeline()
run_pipeline()          # on purpose

rows in the sales table: 107        <-- not 214
```

Running it twice gave the same answer as running it once. That property has a name: the pipeline is **idempotent**. One keyword gives it to us — `if_exists="replace"` throws away the old table and writes a fresh one each run.

Watch out

The most common bug in a first pipeline.

```
out.to_sql("sales", con, if_exists="append", index=False)    # DON'T
```

> With `append`, every run adds another copy. Run it three times and the café has sold everything three times.

A log line per step

```
  EXTRACT    107 rows read
  TRANSFORM  107 rows cleaned, 8 distinct items
  LOAD       107 rows written, revenue $868.35
```

Three `print` statements. Now when the job runs at 6 a.m. and something looks odd, you can see *which step* the rows disappeared at instead of guessing.

A check before you publish

An **assertion** is a sentence that must be true. If it is not, Python stops immediately and tells you why.

```
assert clean["category"].notna().all(), "some items did not match the menu
    "
assert (clean["revenue"] > 0).all(),    "a sale has zero or negative
    revenue"
```

Where the checks go

Between Transform and Load. That is the last moment before the numbers reach somebody else. If a check fails, the pipeline stops *before* writing to the database, and the manager's report is still yesterday's — which is correct, and much better than today's report being wrong.

5. The same thing in SQL

Once the data is in the database, the summary is one query.

```
SELECT sale_date, category,
       SUM(qty)               AS items_sold,
       ROUND(SUM(revenue), 2) AS revenue
FROM   sales
GROUP  BY sale_date, category
ORDER  BY sale_date, category;
```

Use pandas when	Use SQL when
reading files and cleaning text	the data is already in a database
the step is awkward to express in SQL	the data is too big for memory
you are still exploring	the question is a standard group-and-total

Most real pipelines use both: pandas to clean and load, SQL to query afterwards. That is exactly what we did.

6. Five things to remember

1. **Extract changes nothing.** Keep the raw data raw, so you can always start over.
2. **Clean before you join.** `"latte"` does not match `"Latte"`.
3. **Check the row count after every join.** Same number in, same number out.
4. **Make it runnable twice.** `if_exists="replace"`, not `"append"`.
5. **Put the checks between Transform and Load** — the last moment before the numbers reach somebody else.

> A pipeline is just **three functions and a schedule**. The difference between a pipeline and a notebook is that somebody else can run it.

7. In-class activity: build a basic data pipeline

Notebook: `ETL_Activity.ipynb` • **Work in pairs** • **About 35 minutes**

The situation

Founders Library records every book checkout and sends the circulation office **one file per day**. Three days have arrived. The office wants a clean table they can query, plus a daily summary of how many books went out by genre.

Your job: build the ETL pipeline. Same five components as the café example — only the data is different.

Source	`activity_data/checkouts_2027-03-0{2,3,4}.csv` and `books.csv`
Extract	read the three daily files and stack them
Transform	fix `loan_days`, clean `patron_type`, join `books`, add a due date
Load	write to `library.db` and `checkout_summary.csv`
Schedule	(we will just talk about this one)

The tasks

#	Time	Task	Expected result
1	5 min	**Extract** — write a function that reads all three daily files and stacks them	82 rows
2	5 min	**Transform 1** — turn `"14 days"` into the number `14`; convert `checkout_date` to a datetime	dtype `int64`, not `object`
3	5 min	**Transform 2** — make `patron_type` consistent	**16 spellings become 4**
4	7 min	**Transform 3** — left-join `books` on `book_id`; compute `due_date = checkout_date + loan_days`	still 82 rows, 0 unmatched
5	6 min	**Load** — build the daily summary; write to `library.db` and a CSV	82 rows loaded
6	8 min	**Put it together** — one `run_pipeline()` function, a log line per step, and **two assertions** between Transform and Load	runs twice, still 82 rows

Bonus: query the database — which genre was checked out most? How many checkouts per patron type?

What to look for while they work

Watch out

Task 3 before Task 4. A pair who joins first and cleans second will get the right row count and the wrong idea. Ask them: *"why did this step have to come before the join?"*

Watch out

Task 6, running it twice. A pair who used `if_exists="append"` will see **246 rows** instead of 82. This is the single most valuable mistake in the activity — let them find it, then ask what the library director would conclude from a report saying the collection circulated three times over.

Watch out

`pd.to_timedelta` needs a unit. For the due date: `pd.to_timedelta(df["loan_days"], unit="D")`. Without `unit="D"` it will not know whether you meant days, hours or nanoseconds.

Three questions to close on

1. **Scheduling.** Your pipeline reads every file in the folder. Tomorrow a fourth file arrives. What happens when you run `run_pipeline()` again — and is that what you want?
2. **Where checks belong.** You put two assertions between Transform and Load. What would go wrong if you put them at the very end, after `load()` had already run?
3. **One thing you would add.** You have a working pipeline. Name one thing it still needs before you would be comfortable letting it run unattended every morning.

What they built: `extract()` → transform → `assert` → `load()` — 82 rows, 2 checks, and a result somebody else can use. Four functions and two checks. That is a data pipeline.

Every figure in this document is produced by the accompanying notebooks, which run top to bottom from a clean kernel. The data are synthetic, generated by `make_data.py` with a fixed seed, and describe no real café, library or patron.